

بسم الله الرحمن الرحيم

King Abdulaziz University  
Faculty of Computing and information Technology  
Computer Science Department



# CPCS-214 Computer Architecture and Organization

## 29 - Floating Point Instructions

# Introduction

- MIPS supports IEEE 754 single precision and double precision formats

# Floating-Point Instructions

- *Addition: single (add.s), double (add.d)*
- *Subtraction: single (sub.s), double (sub.d)*
- *Multiplication: single (mul.s), double (mul.d)*
- *Division: single (div.s), double (div.d)*
- *Comparison: single (c.x.s) and, double (c.x.d)*
  - where x: = (eq), <> (neq), < (lt), ≤ (le), > (gt), or ≥ (ge)
- *Branch: true (bc1t) and, false (bc1f)*

# Floating-Point Registers

- MIPS designers decided to add separate FP registers
  - Called `$f0`, `$f1`, `$f2`, ...
  - Used either for single precision or double precision
  - Included separate loads and stores for FP registers: `lwc1` and `swc1`
  - Base registers for FP data transfers which are used for addresses remain integer registers
  - Unlike integer registers, FP register 0 can contain a number

# Example

- Write code to load two single precision numbers from memory, add them, and then store sum.

# Example

- Write code to load two single precision numbers from memory, add them, and then store sum.

```
lwc1 $f4,c($sp) # Load 32-bit FP number into F4
lwc1 $f6,a($sp) # Load 32-bit FP number into F6
add.s $f2,$f4,$f6 # F2 = F4 + F6 single precision
swc1 $f2,b($sp) # Store 32-bit FP number from F2
```

# Double Precision Registers

- It is 64-bits
- A double precision register is an *even-odd* pair of single precision registers, using the even register number as its name
- For example, the pair of single precision registers `$f2` and `$f3` form double precision register named `$f2`
- MIPS-32 added `l.d` and `s.d` to the instruction set to load and store 64-bits registers

# Operands

## MIPS floating-point operands

| Name                        | Example                                                | Comments                                                                                                                                                                                                                      |
|-----------------------------|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 32 floating-point registers | \$f0, \$f1, \$f2, . . . , \$f31                        | MIPS floating-point registers are used in pairs for double precision numbers.                                                                                                                                                 |
| $2^{30}$ memory words       | Memory[0],<br>Memory[4], . . . ,<br>Memory[4294967292] | Accessed only by data transfer instructions. MIPS uses byte addresses, so sequential word addresses differ by 4. Memory holds data structures, such as arrays, and spilled registers, such as those saved on procedure calls. |



# Assembly Language

## MIPS floating-point assembly language

| Category           | Instruction                           | Example              | Meaning                                         | Comments                              |
|--------------------|---------------------------------------|----------------------|-------------------------------------------------|---------------------------------------|
| Arithmetic         | FP add single                         | add.s \$f2,\$f4,\$f6 | $\$f2 = \$f4 + \$f6$                            | FP add (single precision)             |
|                    | FP subtract single                    | sub.s \$f2,\$f4,\$f6 | $\$f2 = \$f4 - \$f6$                            | FP sub (single precision)             |
|                    | FP multiply single                    | mul.s \$f2,\$f4,\$f6 | $\$f2 = \$f4 \times \$f6$                       | FP multiply (single precision)        |
|                    | FP divide single                      | div.s \$f2,\$f4,\$f6 | $\$f2 = \$f4 / \$f6$                            | FP divide (single precision)          |
|                    | FP add double                         | add.d \$f2,\$f4,\$f6 | $\$f2 = \$f4 + \$f6$                            | FP add (double precision)             |
|                    | FP subtract double                    | sub.d \$f2,\$f4,\$f6 | $\$f2 = \$f4 - \$f6$                            | FP sub (double precision)             |
|                    | FP multiply double                    | mul.d \$f2,\$f4,\$f6 | $\$f2 = \$f4 \times \$f6$                       | FP multiply (double precision)        |
|                    | FP divide double                      | div.d \$f2,\$f4,\$f6 | $\$f2 = \$f4 / \$f6$                            | FP divide (double precision)          |
| Data transfer      | load word copr. 1                     | lwc1 \$f1,100(\$s2)  | $\$f1 = \text{Memory}[\$s2 + 100]$              | 32-bit data to FP register            |
|                    | store word copr. 1                    | swc1 \$f1,100(\$s2)  | $\text{Memory}[\$s2 + 100] = \$f1$              | 32-bit data to memory                 |
| Conditional branch | branch on FP true                     | bclt 25              | if (cond == 1) go to PC + 4 + 100               | PC-relative branch if FP cond.        |
|                    | branch on FP false                    | bclf 25              | if (cond == 0) go to PC + 4 + 100               | PC-relative branch if not cond.       |
|                    | FP compare single (eq,ne,lt,le,gt,ge) | c.lt.s \$f2,\$f4     | if ( $\$f2 < \$f4$ )<br>cond = 1; else cond = 0 | FP compare less than single precision |
|                    | FP compare double (eq,ne,lt,le,gt,ge) | c.lt.d \$f2,\$f4     | if ( $\$f2 < \$f4$ )<br>cond = 1; else cond = 0 | FP compare less than double precision |

# Machine Language

## MIPS floating-point machine language

| Name       | Format | Example |        |        |        |        |        | Comments                      |
|------------|--------|---------|--------|--------|--------|--------|--------|-------------------------------|
| add.s      | R      | 17      | 16     | 6      | 4      | 2      | 0      | add.s \$f2,\$f4,\$f6          |
| sub.s      | R      | 17      | 16     | 6      | 4      | 2      | 1      | sub.s \$f2,\$f4,\$f6          |
| mul.s      | R      | 17      | 16     | 6      | 4      | 2      | 2      | mul.s \$f2,\$f4,\$f6          |
| div.s      | R      | 17      | 16     | 6      | 4      | 2      | 3      | div.s \$f2,\$f4,\$f6          |
| add.d      | R      | 17      | 17     | 6      | 4      | 2      | 0      | add.d \$f2,\$f4,\$f6          |
| sub.d      | R      | 17      | 17     | 6      | 4      | 2      | 1      | sub.d \$f2,\$f4,\$f6          |
| mul.d      | R      | 17      | 17     | 6      | 4      | 2      | 2      | mul.d \$f2,\$f4,\$f6          |
| div.d      | R      | 17      | 17     | 6      | 4      | 2      | 3      | div.d \$f2,\$f4,\$f6          |
| lwc1       | I      | 49      | 20     | 2      | 100    |        |        | lwc1 \$f2,100(\$s4)           |
| swc1       | I      | 57      | 20     | 2      | 100    |        |        | swc1 \$f2,100(\$s4)           |
| bclt       | I      | 17      | 8      | 1      | 25     |        |        | bclt 25                       |
| bclt.f     | I      | 17      | 8      | 0      | 25     |        |        | bclt.f 25                     |
| c.l.t.s    | R      | 17      | 16     | 4      | 2      | 0      | 60     | c.l.t.s \$f2,\$f4             |
| c.l.t.d    | R      | 17      | 17     | 4      | 2      | 0      | 60     | c.l.t.d \$f2,\$f4             |
| Field size |        | 6 bits  | 5 bits | 5 bits | 5 bits | 5 bits | 6 bits | All MIPS instructions 32 bits |

# Example

- C/Java code to convert a temperature in Fahrenheit to Celsius:

```
float f2c (float fahr)
{
    return ((5.0/9.0) *(fahr - 32.0));
}
```

- Assume that floating-point argument `fahr` is passed in `$f12` and result should go in `$f0`. What is the assembly code?

## Example (cont.)

- Assume that the compiler places the three FP constants in memory within easy reach of the global pointer \$gp.

```
f2c: lwc1    $f16, const5($gp)    # f16 = 5  
     lwc1    $f18, const9($gp)    # f18 = 9
```

```
float f2c (float fahr)  
{  
    return ((5.0/9.0)*(fahr-32.0));  
}
```

## Example (cont.)

- Assume that the compiler places the three FP constants in memory within easy reach of the global pointer \$gp.

```
f2c: lwc1    $f16, const5($gp)    # f16 = 5
     lwc1    $f18, const9($gp)    # f18 = 9
     div.s   $f16, $f16, $f18      # f16 = 5/9
```

```
float f2c (float fahr)
{
    return ((5.0/9.0)*(fahr-32.0));
}
```

## Example (cont.)

- Assume that the compiler places the three FP constants in memory within easy reach of the global pointer \$gp.

```
f2c: lwc1    $f16, const5($gp)    # f16 = 5
     lwc1    $f18, const9($gp)    # f18 = 9
     div.s   $f16, $f16, $f18     # f16 = 5/9
     lwc1    $f18, const32($gp)   # f18 = 32
```

```
float f2c (float fahr)
{
    return ((5.0/9.0)*(fahr-32.0));
}
```

## Example (cont.)

- Assume that the compiler places the three FP constants in memory within easy reach of the global pointer \$gp.

```
f2c: lwc1    $f16, const5($gp)    # f16 = 5
     lwc1    $f18, const9($gp)    # f18 = 9
     div.s   $f16, $f16, $f18     # f16 = 5/9
     lwc1    $f18, const32($gp)   # f18 = 32
     sub.s   $f18, $f12, $f18     # f18 = fahr-32
     mul.s   $f0, $f16, $f18     # f0 = result
     jr      $ra                 # return
```

```
float f2c (float fahr)
{
    return ((5.0/9.0)*(fahr-32.0));
}
```